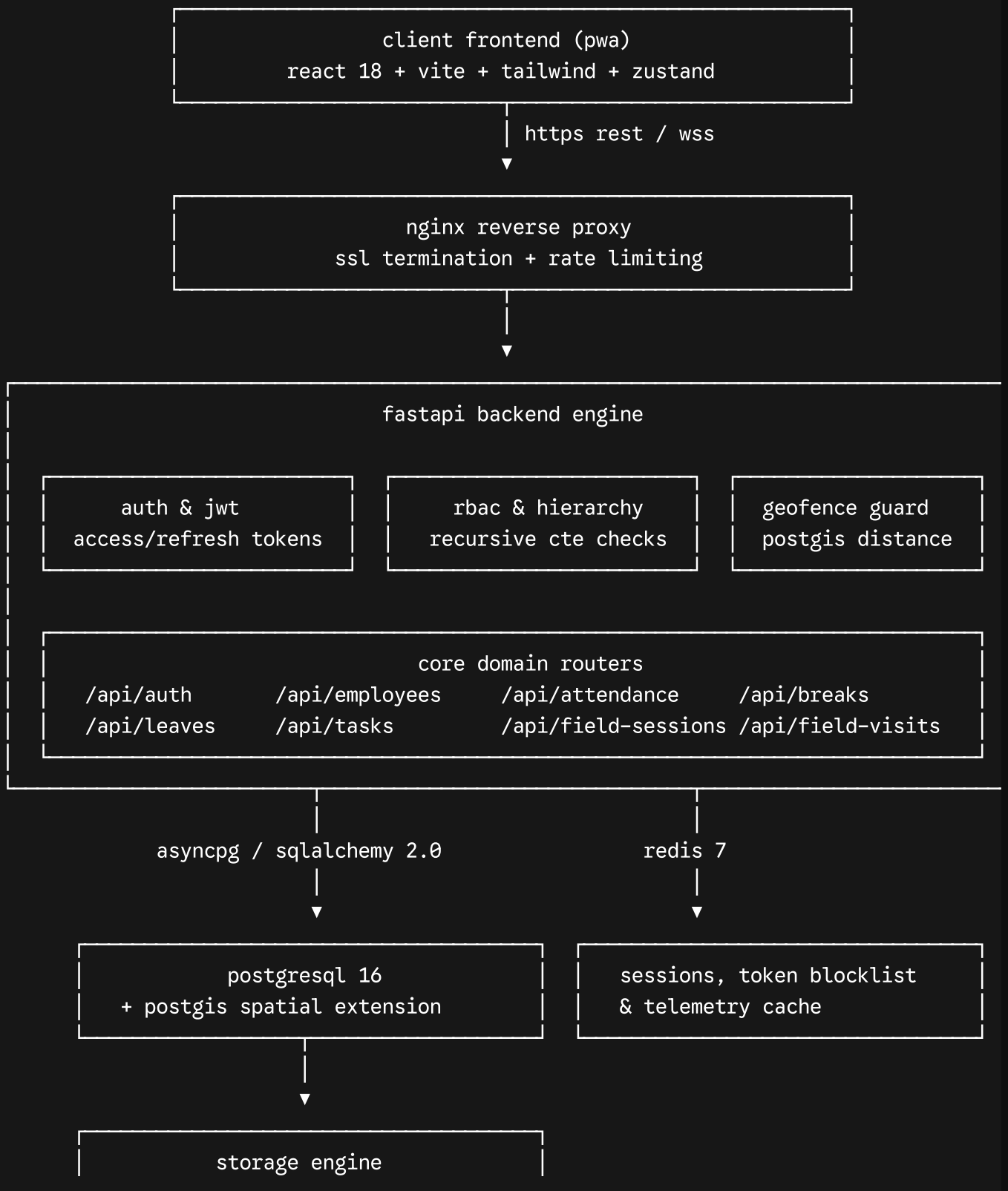
fsm core foundation: architecture & engineering specification

1. high-level system architecture



```
        │   watermarked check-in proofs / s3   │
        └──────────────────────────────────────┘
```

2. database layer: core postgresql schema

the baseline schema provides data persistence for core workforce interactions without relying on commercial crm tables:

- `pharma_hrms_employees` : stores user identity, credentials hash, reporting lines via `reporting_to`, corporate rank, and assigned operational territory.
  MD + 1
- `pharma_hrms_refresh_tokens` : stores hashed refresh tokens, device fingerprints, and expiration dates for secure session rotation.
  MD
- `pharma_hrms_office_locations` : stores approved geofenced locations, coordinates, radius boundaries, and postgis spatial geometry points.
  MD + 1
- `pharma_hrms_attendance` : stores daily punch logs, in and out timestamps, punch types, verification status, and attendance classifications.
  MD + 1
- `pharma_hrms_breaks` : tracks work pause intervals throughout the day with start and end timestamps.
  MD
- `pharma_hrms_leave_requests` : stores leave applications, date ranges, reasons, and approval workflow states.
  MD + 1
- `pharma_hrms_leave_balances` : stores allocated, used, and remaining leave counts per employee per leave category.
  MD + 1
- `pharma_hrms_field_sessions` : tracks start and end of travel sessions, accumulated distance, and day-level operational status.
  MD + 1
- `pharma_hrms_field_visits` : tracks scheduled stops, target entity details, lat-long coordinates, notes, and proof timestamps.
  MD + 1
- `pharma_hrms_tasks` : stores assignments delegated by managers, due dates, statuses, and completion timestamps.
  MD + 1
- `pharma_hrms_audit_log` : immutable log capturing actor id, action, changed resource, and before and after state snapshots.
  MD

3. backend service layer: fastapi

the core backend decouples database transactions from route handlers through five core services:

a. auth & security service

- executes bcrypt password verification.
  MD
- generates short-lived access tokens and longer-lived refresh tokens.
  MD
- verifies active employee status and extracts user identity before downstream processing.
  MD

b. organizational hierarchy & rbac service

- executes recursive common table expressions over the employee reporting tree to determine all subordinates belonging to a given manager.
  MD + 1
- enforces boundary isolation so managers only query data from their downward reporting lines.
  MD + 1

c. spatial geofencing & verification service

- runs server-side postgis spatial queries to verify user coordinates against allowed geofence boundaries.
  MD + 1
- flags spoofed coordinates, excessive speed between telemetry pings, and degraded accuracy readings.
  MD + 1

d. transactional leave service

- runs balance validation, request state transition, balance deduction, and calendar day attendance tagging inside a single atomic database transaction.
  MD

e. automated background scheduler

- runs daily jobs using an asynchronous scheduler to close out past-due pending tasks automatically.
  MD + 1

4. core api routing structure

the core foundation exposes these standard endpoints:

Plaintext

```
/api/auth
  POST   /login                authenticate and return token pair
  POST   /refresh              rotate expired access token[cite: 2]
  POST   /logout               invalidate current session[cite: 2]

/api/employees
  GET    /me                   return authenticated user profile[cite: 2]
  GET    /directory            list employees scoped by reporting tree[cite: 2]
  GET    /org-chart            hierarchical reporting visualization[cite: 2]

/api/attendance
  POST   /clock-in             punch in with coordinates and proof image
  POST   /clock-out            punch out with end-of-day summary
  GET    /today                current day attendance and active timers[cite: 2]

/api/breaks
  POST   /start                pause active working session[cite: 2]
  POST   /end                  resume active working session[cite: 2]

/api/field-sessions
  POST   /start                begin day travel and location tracking
  POST   /{id}/end             close session and compute distance[cite: 1, 2]

/api/field-visits
  GET    /                     list assigned visits for date[cite: 2]
  PATCH  /{id}/start           mark arrival and upload proof[cite: 1, 2]
  PATCH  /{id}/complete        mark finish with discussion notes[cite: 1, 2]

/api/leaves
  POST   /apply                submit leave application[cite: 1, 2]
  GET    /my-requests          list personal leave history[cite: 2]
  PATCH  /{id}/approve         manager approval with atomic deduction[cite: 1, 2]

/api/tasks
  GET    /                     list pending and active tasks[cite: 2]
  POST   /                     delegate task to subordinate[cite: 1, 2]
  PATCH  /{id}/status          update progress state[cite: 2]
```

5. frontend foundation shell: react pwa

the frontend shell manages global state and hardware integration[cite: 1, 2]:

- **authentication context:** manages token persistence, automatic token renewal, and current user profile state[cite: 2].
- **app shell:** provides responsive desktop sidebar, mobile bottom navigation, and global layout wrappers.
  MD
- **camera and geolocation hook:** captures camera frames directly from device hardware and acquires current gps coordinates[cite: 1, 2].
- **network and telemetry sync:** caches punch and visit events locally in offline storage and flushes them sequentially with idempotency keys once connectivity returns[cite: 1, 2].

6. backend repository layout

Plaintext

```
fsm_core/
├── app/
│   ├── config.py                # application settings and environment validation[c
│   ├── main.py                  # core application factory mounting base routers[ci
│   ├── database/
│   │   ├── engine.py            # async connection pool[cite: 2]
│   │   └── session.py           # database session dependency[cite: 2]
│   ├── models/                  # core sqlalchemy models[cite: 2]
│   │   ├── employee.py[cite: 2]
│   │   ├── attendance.py[cite: 2]
│   │   ├── break_record.py[cite: 2]
│   │   ├── leave_request.py[cite: 2]
│   │   ├── leave_balance.py[cite: 2]
│   │   ├── task.py[cite: 2]
│   │   ├── field_session.py[cite: 2]
│   │   ├── field_visit.py[cite: 2]
│   │   ├── office_location.py[cite: 2]
│   │   └── audit_log.py[cite: 2]
│   ├── routers/                 # core domain routers[cite: 2]
│   │   ├── auth.py[cite: 2]
│   │   ├── employees.py[cite: 2]
│   │   ├── attendance.py[cite: 2]
│   │   ├── breaks.py[cite: 2]
│   │   ├── leaves.py[cite: 2]
│   │   ├── tasks.py[cite: 2]
│   │   └── field_visits.py[cite: 2]
│   ├── services/                # core domain business logic[cite: 2]
│   │   ├── auth_service.py[cite: 2]
│   │   ├── hierarchy_service.py[cite: 2]
│   │   ├── geofence_service.py[cite: 2]
```

```
│   │   └── leave_service.py[cite: 2]
│   └── dependencies/
│       ├── auth.py               # token authentication guards[cite: 2]
│       └── permissions.py        # hierarchy role guards[cite: 2]
```

Any modular extension built later, such as lead pipelines or expense claims, mounts on top of this foundation package without modifying baseline code.